

Optimising Numerical Code



Suman 7 Feb, 2026 At 8:00 PM Notes Recommended Resource

Optimising Numerical Code



Associated Lectures (1)



Description



Discussions

Session 3.3: Optimising Numerical Code & Introduction to Pandas

In-Class Resources

Lecture Notes - Quick Reference Guide

Program: AIM101 - Artificial Intelligence & Machine Learning Foundation **Institution:** Vishlesan i-Hub IIT Patna x Masai School **Session Type:** Bridge Session (NumPy to Pandas) **Prerequisites:** Sessions 3.1-3.2 (NumPy Fundamentals and Power Tools)

Session Overview

This bridge session covers three essential topics:

Part	Topic	Duration	Focus
Part 1	List Comprehension	30 min	Full Teaching
Part 2	Performance & Optimization	15 min	Condensed Wrap-up
Part 3	Introduction to Pandas	45 min	Conceptual Foundation

PART 1: LIST COMPREHENSION

1.1 Basic Syntax

String Transformations

```
names = ["alice", " BOB ", "Charlie"]

# Uppercase and strip whitespace
clean = [name.strip().upper() for name in names] # ['ALICE', 'BOB', 'CHARLIE']

# Get first letter of each
initials = [name.strip()[0].upper() for name in names] # ['A', 'B', 'C']

# Get lengths
lengths = [len(name.strip()) for name in names] # [5, 3, 7]
```

1.3 Filtering with Conditions

Syntax for Filtering

```
[expression for item in iterable if condition]
```

Key Point: The `if` clause comes AFTER the `for`. It acts as a filter - only items passing the condition are included.

Filtering Examples

```
numbers = [-5, -2, 0, 3, 7, -1, 8, 12]

# Only positive numbers
positives = [x for x in numbers if x > 0] # [3, 7, 8, 12]

# Only even numbers
evens = [x for x in numbers if x % 2 == 0] # [-2, 0, 8, 12]

# Multiple conditions (AND)
pos_even = [x for x in numbers if x > 0 and x % 2 == 0] # [8, 12]

# Filter AND transform
pos_squares = [x ** 2 for x in numbers if x > 0] # [9, 49, 64, 144]
```

String Filtering

```
words = ["apple", "banana", "cherry", "date", "elderberry"]

# Words longer than 5 characters
long_words = [w for w in words if len(w) > 5] # ['banana', 'cherry', 'elderberry']

# Words starting with vowels
vowel_words = [w for w in words if w[0] in 'aeiou'] # ['apple', 'elderberry']
```

```
# Words containing 'a'
with_a = [w for w in words if 'a' in w] # ['apple', 'banana', 'date']
```

1.4 Conditional Expressions (if-else)

Two Types of Conditions - IMPORTANT DISTINCTION

```
# TYPE 1: FILTERING (if at END) - list gets SHORTER
[x for x in items if condition]

# TYPE 2: TRANSFORMING (if-else BEFORE for) - list stays SAME LENGTH
[a if condition else b for x in items]
```

Comparison Example

```
numbers = [1, 2, 3, 4, 5, 6]

# Filtering: only keeps even numbers (list gets shorter)
filtered = [x for x in numbers if x % 2 == 0]
# Result: [2, 4, 6], length: 3

# Transforming: keeps all, but labels them (same length)
transformed = ["even" if x % 2 == 0 else "odd" for x in numbers]
# Result: ['odd', 'even', 'odd', 'even', 'odd', 'even'], length: 6
```

Practical if-else Examples

```
values = [-3, 5, -1, 8, -2, 10]

# Replace negatives with 0
non_negative = [x if x >= 0 else 0 for x in values] # [0, 5, 0, 8, 0, 10]

# Grade classification
marks = [45, 78, 32, 91, 55]
results = ["Pass" if m >= 40 else "Fail" for m in marks]
# ['Pass', 'Pass', 'Fail', 'Pass', 'Pass']
```

1.5 Dictionary, Set, and Generator Comprehensions

Syntax Cheat Sheet

Type	Syntax	Example
List	[expr for x in iterable]	[x**2 for x in range(5)]

Type	Syntax	Example
Dict	<code>{key: value for x in iterable}</code>	<code>{x: x**2 for x in range(5)}</code>
Set	<code>{expr for x in iterable}</code>	<code>{x % 3 for x in range(10)}</code>
Generator	<code>(expr for x in iterable)</code>	<code>(x**2 for x in range(5))</code>

Dictionary Comprehension

```
# Basic: number to square mapping
squares_dict = {x: x**2 for x in range(1, 6)}      # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# From two lists using zip
names = ['Alice', 'Bob', 'Charlie']
ages = [25, 30, 35]
name_to_age = {name: age for name, age in zip(names, ages)}
# {'Alice': 25, 'Bob': 30, 'Charlie': 35}

# Creating a lookup table
products = [
    {'id': 'P001', 'name': 'Laptop', 'price': 1200},
    {'id': 'P002', 'name': 'Phone', 'price': 800}
]
product_lookup = {p['id']: p for p in products}
# Access: product_lookup['P002'] -> {'id': 'P002', 'name': 'Phone', 'price': 800}

# Filtering a dictionary
prices = {'apple': 1.20, 'banana': 0.50, 'mango': 2.50}
cheap = {item: price for item, price in prices.items() if price < 1.00}
# {'banana': 0.50}
```

Set Comprehension

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Unique remainders when dividing by 3
remainders = {x % 3 for x in numbers}              # {0, 1, 2}

# Unique first letters
words = ['apple', 'apricot', 'banana', 'blueberry']
first_letters = {w[0] for w in words}               # {'a', 'b'}
```

Generator Expression

```
import sys

# List: stores all values in memory
list_comp = [x**2 for x in range(1000)]
print(sys.getsizeof(list_comp))    # ~8856 bytes

# Generator: produces values on-demand
gen_exp = (x**2 for x in range(1000))
```

```
print(sys.getsizeof(gen_exp))    # ~200 bytes (much smaller!)

# Use generators with aggregation functions
total = sum(x**2 for x in range(1000)) # No extra brackets needed inside sum()
```

Rule of thumb: If you're only iterating once, use a generator.

1.6 Best Practices & Common Mistakes

When NOT to Use Comprehensions

- Logic is too complex (more than 2 conditions)
- You need multiple statements per iteration
- Readability suffers
- You need to handle exceptions

Golden Rule: "If it doesn't fit on one line or is hard to understand at a glance, use a regular loop."

Common Mistakes to Avoid

Mistake	Why It's Bad
Over-nesting (>2 levels)	Unreadable
Confusing <code>if</code> position	Filter vs transform confusion
Large comprehensions	Memory issues (use generators)
Side effects	<code>[print(x) for x in items]</code> creates useless list

Bad vs Good

```
# BAD: Too complex
result = [x * y for x in range(10) if x % 2 == 0 for y in range(10) if y % 3 == 0]

# GOOD: Use a loop for complex logic
result = []
for x in range(10):
    if x % 2 == 0:
        for y in range(10):
            if y % 3 == 0:
                result.append(x * y)

# BAD: Comprehension for side effects
[print(x) for x in range(5)] # Creates list of Nones

# GOOD: Use a loop for side effects
for x in range(5):
    print(x)
```

PART 2: PERFORMANCE & OPTIMIZATION

2.1 The Performance Hierarchy

Speed Ladder (Slowest to Fastest)

FASTEST

- ^ C/Cython (100x+)
- ^ Numba JIT (10-50x)
- ^ NumPy Vectorized (10-100x)
- ^ List Comprehension (1.1-1.3x)
- ^ Python for loop (1x baseline)

SLOWEST

When to Use What

Situation	Best Tool
Small data, mixed types	List comprehension
Large numerical arrays	NumPy vectorization
Can't vectorize, need speed	Numba @jit
Building a library	Cython (advanced)

Key insight: Most of the time, NumPy vectorization is enough!

2.2 Performance Comparison Example

```
import numpy as np

N = 100000

# Method 1: Python for loop (~10-15 ms)
def using_loop(n):
    total = 0
    for i in range(n):
        total += i ** 2
    return total

# Method 2: List comprehension (~8-12 ms)
def using_listcomp(n):
    return sum([i ** 2 for i in range(n)])

# Method 3: Generator expression (~10-14 ms, less memory)
```

```
def using_generator(n):  
    return sum(i ** 2 for i in range(n))  
  
# Method 4: NumPy vectorized (~0.1-0.3 ms) -- 50-100x FASTER!  
def using_numpy(n):  
    arr = np.arange(n)  
    return np.sum(arr ** 2)
```

NumPy's dramatic speedup comes from:

- Compiled C code under the hood
- No Python interpreter overhead per element
- Contiguous memory access patterns

2.3 Vectorization Patterns Quick Reference

Common Loop-to-NumPy Replacements

Loop Pattern	NumPy Replacement
Accumulator (<code>sum += x</code>)	<code>np.sum()</code> , <code>np.cumsum()</code>
Conditional check	Boolean masking
Element-wise math	Vectorized operations
Nested loops (i, j)	Broadcasting

Code Patterns

```
import numpy as np  
  
# Pattern 1: Accumulator -> np.sum()  
# SLOW  
total = 0  
for x in data:  
    total += x  
# FAST  
total = np.sum(data)  
  
# Pattern 2: Conditional -> Boolean masking  
# SLOW  
result = []  
for x in data:  
    if x > 0:  
        result.append(x)  
# FAST  
result = data[data > 0]  
  
# Pattern 3: Nested loops -> Broadcasting  
# SLOW  
result = []
```



```
for i in range(len(a)):
    for j in range(len(b)):
        result.append(a[i] + b[j])
# FAST
result = a[:, np.newaxis] + b
```

2.4 The Knuth Rule

"Premature optimization is the root of all evil." - Donald Knuth (Turing Award winner)

The Three Steps

First: Make it WORK (correct behavior)

Second: Make it READABLE (maintainable code)

Third: Make it FAST (only if needed!)

Remember: Profile before you optimize!

2.5 Numba & Cython (Awareness Level)

Numba

- Add `@jit` decorator to speed up numerical Python
- Great for loops that can't be vectorized
- Example: Custom numerical algorithms

Cython

- Python with C type declarations
- Used internally by pandas, scikit-learn
- For library developers, not daily use

Message: Know these exist. Use NumPy first. These are for when you REALLY need more speed.

PART 3: INTRODUCTION TO PANDAS

3.1 Why Pandas Exists

The Tabular Data Problem

- Most business data is TABLES (like Excel spreadsheets)

- Tables have: Column names, Row identifiers, Mixed data types
- NumPy arrays: Only positions, no labels, same dtype

NumPy vs Pandas Access

```
# NumPy: What's in position [0, 2]?
data[0, 2] # Returns 50000 – but what IS this?

# Pandas: What's Alice's Salary?
data.loc['Alice', 'Salary'] # Returns 50000 – self-documenting!
```

3.2 NumPy vs Pandas Decision Guide

Use NumPy When...	Use Pandas When...
Pure numerical computation	Tabular data with mixed types
Matrix operations, linear algebra	Need labeled columns/rows
Performance-critical numerical code	Reading CSV/Excel files
Working with images, signals	Data cleaning, exploration
Homogeneous data types	Heterogeneous data types

Key Insight: Pandas is BUILT ON NumPy. DataFrames store data as NumPy arrays internally!

The Analogy

- **NumPy** = Scientific Calculator (fast math on numbers)
- **Pandas** = Excel/Google Sheets (structured, labeled data)

3.3 The Pandas Data Model

Series: 1D Labeled Array

```
import pandas as pd

# NumPy array – position only
np.array([100, 200, 150]) # Index: 0, 1, 2

# Pandas Series – labeled
sales = pd.Series([100, 200, 150],
                  index=['Jan', 'Feb', 'Mar'],
                  name='Monthly Sales')

# Index: Jan, Feb, Mar
```

```
# Access by label
sales['Feb']    # Returns 200

# Access by position (still works!)
sales[1]        # Returns 200
```

Analogy: A single column from a spreadsheet, with row labels.

DataFrame: 2D Labeled Structure

```
# Create DataFrame from dictionary
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Salary': [50000, 60000, 70000]
})
```

Output:

	Name	Age	Salary
0	Alice	25	50000
1	Bob	30	60000
2	Charlie	35	70000

Think of it as: Dictionary of Series sharing the same index.

The Hierarchy

```
NumPy Array
  |
  v
Pandas Series (1D labeled)
  |
  v
Pandas DataFrame (2D labeled = collection of Series)
```

- Each DataFrame column is a Series
- Each Series is backed by a NumPy array

3.4 Creating DataFrames

Method 1: From Dictionary (Most Common)

```
products = pd.DataFrame({
    'product': ['Laptop', 'Phone', 'Tablet'],
    'price': [1200, 800, 400],
})
```

```
'quantity': [10, 25, 15]
})
```

Method 2: From List of Lists

```
data = [
    ['Alice', 25, 'Engineering'],
    ['Bob', 30, 'Sales']
]
df = pd.DataFrame(data, columns=['Name', 'Age', 'Department'])
```

Method 3: From NumPy Array

```
df = pd.DataFrame(
    np.random.rand(3, 2),
    columns=['X', 'Y']
)
```

3.5 Quick Exploration Methods (ESSENTIAL)

The First Commands to Run on Any DataFrame

```
df.head()      # First 5 rows (use head(10) for 10 rows)
df.tail()      # Last 5 rows
df.shape       # (rows, columns) as tuple
df.columns     # Column names
df.dtypes     # Data type of each column
df.info()      # Complete summary (dtypes, non-null counts, memory)
df.describe()  # Statistical summary (count, mean, std, min, max, quartiles)
```

Example Output

```
products.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   product     3 non-null      object
 1   price       3 non-null      int64
 2   quantity    3 non-null      int64
dtypes: int64(2), object(1)
memory usage: 200.0+ bytes
```

```
products.describe()
```

	price	quantity
count	3.00	3.00
mean	800.00	16.67
std	400.00	7.64
min	400.00	10.00
25%	600.00	12.50
50%	800.00	15.00
75%	1000.00	20.00
max	1200.00	25.00

Always start here! Know your data before doing anything else.

3.6 NumPy to Pandas Transition Example

```
# Start with NumPy array
sales_data = np.array([
    [100, 150, 200],
    [120, 180, 220],
    [90, 160, 190]
])

# NumPy access (position-based)
sales_data[0, 0] # Returns 100 - but what does [0, 0] mean?

# Convert to Pandas with labels
sales_df = pd.DataFrame(
    sales_data,
    index=['Product A', 'Product B', 'Product C'],
    columns=['Q1', 'Q2', 'Q3']
)

# Pandas access (label-based)
sales_df.loc['Product A', 'Q1'] # Returns 100 - self-documenting!
```

Output:

	Q1	Q2	Q3
Product A	100	150	200
Product B	120	180	220
Product C	90	160	190

3.7 Preview: What's Coming in Session 4.1

Topic	What You'll Learn
Loading data	<code>pd.read_csv()</code> , <code>pd.read_excel()</code>
Selecting columns	<code>df['col']</code> , <code>df[['col1', 'col2']]</code>
Selecting rows	<code>df.loc[]</code> , <code>df.iloc[]</code>
Filtering	<code>df[df['col'] > value]</code>
Sorting	<code>df.sort_values()</code>

Today: You learned WHAT Pandas is and WHY you need it. **Session 4.1:** You'll learn HOW to use it.

QUICK REFERENCE CHEAT SHEET

List Comprehension Syntax

```
# Basic
[expression for item in iterable]

# With filter
[expression for item in iterable if condition]

# With conditional transform
[a if condition else b for item in iterable]

# Dictionary
{key: value for item in iterable}

# Set
{expression for item in iterable}

# Generator
(expression for item in iterable)
```

Performance Hierarchy

```
Python loop < List comp < NumPy < Numba < C/Cython
1x          1.1-1.3x   10-100x  10-50x   100x+
```

NumPy vs Pandas

Feature	NumPy	Pandas
Dimensions	N-dimensional	1D (Series), 2D (DataFrame)
Labels	Position only	Named columns/rows
Data types	Homogeneous	Heterogeneous (mixed types)
Best for	Numerical computation	Tabular data, data cleaning

Essential Pandas Methods

```
# First 5 things to run on any DataFrame
df.head()      # View first 5 rows
df.shape       # Get dimensions (rows, cols)
df.columns     # Get column names
df.info()      # Summary with dtypes and nulls
df.describe()  # Statistical summary
```

Key Takeaways

List comprehensions are the Pythonic way to transform and filter data

Filter vs Transform: `if` at end = filter (shorter list); `if-else` before `for` = transform (same length)

Performance hierarchy: Loops < List comp < NumPy < Numba

Knuth's rule: Make it work, make it readable, THEN make it fast

Pandas exists because real data has labels and mixed types

Series = 1D labeled array; **DataFrame** = 2D labeled structure

Always explore first: `head()`, `info()`, `describe()`

Footer: Vishlesan i-Hub IIT Patna x Masai School